

# Calling vetted calculation modules from a job file

---

A simply supported roof beam, analysed over the ultimate load combinations of AS/NZS 1170 and designed for flexure and shear to AS 3600:2018 — written the way you would run a real job.

|                |                                                                                                                                                |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| WRITTEN FOR    | An engineer who reads design codes fluently and Python slowly. No prior Python project experience assumed.                                     |
| LIBRARY        | <code>austruct</code> 0.1.0 — the toolkit in this repository                                                                                   |
| WORKED EXAMPLE | Job 25-0142 Northbank Depot, roof beam B1                                                                                                      |
| FRAMEWORK      | The six ASET components (Ferster, 2025)                                                                                                        |
| EVERY PANEL    | Real output. Each terminal panel is the captured stdout of the command shown; each code panel is read out of the committed file at build time. |
| STATUS         | <b>Not verified for issue.</b> This document teaches the workflow. It makes no claim that any number the library produces is correct.          |

## 0 What you are going to build

By the end of this you will have a folder for one job, containing two short Python files you wrote, that together produce a signed calculation report for a beam. Neither of your files contains a single clause of AS 3600. Every code equation comes from a library that lives somewhere else, is version controlled, and is tested.

That separation is the whole idea, and it is worth being blunt about why it matters. A spreadsheet mixes three things that have completely different lifetimes:

| What it is                                                     | How often it changes                           | Who should own it                                       |
|----------------------------------------------------------------|------------------------------------------------|---------------------------------------------------------|
| The <b>code equations</b> — $\phi M_{u0}$ , $k_v$ , $\theta_v$ | When the Standard is amended. Every few years. | One vetted, tested library. Written once, checked once. |
| The <b>job</b> — spans, loads, grades, the client's name       | Every job. Sometimes every day.                | You, in a job folder.                                   |
| The <b>decision</b> — 350 × 650, 3-N24                         | Every iteration, until it lands.               | You, in one line you can read aloud.                    |

In a spreadsheet all three are tangled in the same cells, so re-using the equations means copying the job with them — and that copy is now a second, unchecked version of AS 3600. In Python you keep them apart by putting the equations in an *importable package* and the job in a *script that imports it*. That is what the word "import" is really doing for you: it is not a convenience, it is the mechanism that stops your calculations from being copied.

### THE SHAPE OF THE FINISHED JOB

Two files you write, one command you run, five files you get back:

- 1 job\_data.py — the job. Project record, spans, tributary widths, areal loads, and the wind pressure worked out to AS/NZS 1170.2. No capacities.
- 2 beam\_B1.py — the member. Says what B1 is, what is on it, and what to check. Every capacity comes from an `import`.
- 3 `python beam_B1.py` — and out come three diagrams, a Markdown report and an HTML report, all stamped with the clause references used to produce them.

### Where this sits in the ASET framework

Ferster's *Anatomy of Your Automated Structural Engineering Toolkit* splits a toolkit into six components. This tutorial walks all six in order, because that order is also the order a real calculation goes in. Keep the map in your head as you go — when you are lost in the code, the question "which of the six am I in?" usually unsticks it.

| # | ASET component          | In this job                                      | Where the code lives                                                                  |
|---|-------------------------|--------------------------------------------------|---------------------------------------------------------------------------------------|
| 1 | Reference data          | C32 concrete, N24 bar areas                      | <code>austruct.materials</code>                                                       |
| 2 | Project data            | Job record → $\psi$ factors → combinations; wind | <code>austruct.project</code> , <code>austruct.loads</code> , <i>your job_data.py</i> |
| 3 | Fast demand calculation | Analyse 7 combinations, envelope them            | <code>austruct.analysis</code>                                                        |
| 4 | Design documentation    | 350 x 650   C32   BOT 3-N24   ...                | <code>austruct.design_documentation</code>                                            |
| 5 | Design verification     | Flexure and shear to AS 3600:2018                | <code>austruct.design.as3600</code>                                                   |
| 6 | Reporting               | The signed audit document                        | <code>austruct.report</code>                                                          |

#### ONE HONEST CAVEAT BEFORE YOU START

Every module in this library currently reports its verification status as `UNVERIFIED`, and every report it produces is stamped **NOT VERIFIED FOR ISSUE**. That is deliberate and it is correct: the constants have been written but not independently checked against the printed Standards. This tutorial teaches you the *workflow*. Checking the numbers is a separate job, and the library is built so that job is possible — which is more than a spreadsheet usually offers.

# 1 How Python finds other people's code

Everything that follows rests on four words: module, package, install, and path. Ten minutes here will save you an afternoon of `ModuleNotFoundError`.

## A module is a file. A package is a folder of them.

A **module** is any `.py` file. `job_data.py` is a module. A **package** is a folder of modules that can be imported as a unit — `austruct` is a package, `austruct.design` is a sub-package inside it, and `austruct.design.as3600.flexure` is a module inside that. The dots in an import statement are folder separators.

So this line:

```
from austruct.design import as3600
```

reads as: *go into the folder `austruct/design/`, and give me the thing called `as3600`*. And once you have it, `as3600.check_flexure(...)` calls a function that lives in a file you did not write, did not copy, and cannot accidentally edit while typing a span.

## The three shapes of an import, and when to use which

| You write                                                     | You then use                                           | Use it w                                                                                |
|---------------------------------------------------------------|--------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>import austruct</code>                                  | <code>austruct.design.as3600.check_flexure(...)</code> | Rarely. T<br>long to i                                                                  |
| <code>from austruct.design import as3600</code>               | <code>as3600.check_flexure(...)</code>                 | <b>Most of<br/>time.</b> Tl<br>still says<br>Standard<br>came fr                        |
| <code>from austruct.design.as3600 import check_flexure</code> | <code>check_flexure(...)</code>                        | Sparingl<br>months<br>check_f<br>alone do<br>tell a re<br>whether<br>was AS<br>or AS 51 |

### NEVER DO THIS

`from austruct.design.as3600 import *` pulls every name in the module into your file at once. It is the Python equivalent of pasting someone's whole spreadsheet into your sheet: you now cannot tell which numbers are yours, and a rename upstream silently changes your results. In a calculation file it is not a style preference, it is a traceability defect.

## Where Python actually looks

When you write `import x`, Python searches a list of folders, in order, and takes the first `x` it finds:

- 1 The folder the script you ran lives in.** Not the folder you are standing in — the folder of the `.py` file you handed to `python`. This is why `beam_B1.py` can say `import job_data` with no setup at all: they are siblings.

- 2 **The site-packages of the environment you are in.** This is where `pip install` puts things, and it is how `austruct` is found from any directory on the machine.
- 3 The standard library, and a few other places you will not need to think about.

Both of the errors below are that list failing, and they read very differently once you know it. First: the library is not in this environment, because the environment was never activated.

```
bash

$ python beam_B1.py          # note: no (.venv) in the prompt
Traceback (most recent call last):
  File "/home/user/EngineeringEngine/tutorials/01_simply_supported_beam/job/beam_B1.py", line 15, in
<module>
    from austruct.analysis import UDL, PointLoad, SelfWeight, analyse_combinations, simply_supported
ModuleNotFoundError: No module named 'austruct'
```

**Screenshot 1.1** — no `(.venv)` in the prompt, so `pip install`'s work is invisible. The fix is `source .venv/bin/activate`, not editing the file.

Second: your own module is not found, because you are running from the wrong directory — and then the same import working one folder down.

```
bash

(.venv) $ cd ~/jobs/25-0142-northbank          # one level above the script
(.venv) $ python -c "import job_data"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ModuleNotFoundError: No module named 'job_data'

(.venv) $ cd job
(.venv) $ python -c "import job_data; print(job_data.SPAN)"
7200.0
```

**Screenshot 1.2** — `import job_data` is not magic. It works because `job_data.py` sits beside the file being run.

#### THE RULE TO MEMORISE

**Library code gets installed. Job code sits next to the script.** If you ever find yourself writing `sys.path.append("../my_calcs")` to reach your own calculations, stop — that is the signal that those calculations have grown up and want to be an installed package too.

## 2 Setting up, from nothing

Two folders, and they must be different folders. One holds the library. One holds the job. Confusing them is the single most common way a Python toolkit turns back into a spreadsheet.

### Step 1 — a virtual environment

A *virtual environment* is a private copy of Python for one project. It exists so that upgrading a package for this job cannot silently change the answers on last year's job. Create it once, in the library folder, and activate it every time you open a terminal. The `(.venv)` that appears in your prompt is the confirmation.

```
bash — in the library folder

$ python3 -m venv .venv
$ source .venv/bin/activate
(.venv) $ pip install -e ".[dev,plots,report,tools]"
  Attempting uninstall: austruct
  Found existing installation: austruct 0.1.0
  Uninstalling austruct-0.1.0:
    Successfully uninstalled austruct-0.1.0
  Successfully installed austruct-0.1.0
```

**Screenshot 2.1** — `pip install -e` installs the library in *editable* mode: `site-packages` gets a pointer to the source folder rather than a copy, so a fix to the library is live immediately. The bracketed names are optional extras — `plots` pulls in matplotlib, `report` pulls in the renderers.

### Step 2 — prove it worked before you write anything

Two commands. The first shows you *which* copy of the library you are about to use, which is the question you will want answered the first time two versions disagree. The second runs the library's own test suite — the evidence that the thing you are about to trust still behaves the way its author intended.

```
bash

(.venv) $ python -c "import austruct; print(austruct.__version__, austruct.__file__)"
0.1.0 /home/user/EngineeringEngine/src/austruct/__init__.py

(.venv) $ pytest -q
799 passed, 3 skipped in 173.67s (0:02:53)
```

**Screenshot 2.2** — the path printed is the source tree, not a copy in `site-packages`: that is what *editable* means. 799 passing tests is not proof the clauses are right, but it is proof nothing has broken since they were written.

#### IF PYTEST FEELS ABSURDLY SLOW

On a CPU-limited container, multi-threaded BLAS spin-waiting inside hundreds of tiny matrix solves can cost two orders of magnitude. Prefix the command with `OPENBLAS_NUM_THREADS=1` `OMP_NUM_THREADS=1`. The 799 tests above ran in under three minutes that way.

### Step 3 — make the job folder

One folder per *job*, not per member. B1, B2 and the transfer slab all share the same project record and the same load schedule, and duplicating those three times is how they drift apart.

```
bash

(.venv) $ tree ~/jobs/25-0142-northbank

25-0142-northbank/          <- ONE folder per job. Not one per beam.
|-- job/
|   |-- job_data.py         <- the job: geometry, loads, wind, the record
|   |-- beam_B1.py          <- the member: analysis + design + report
|-- outputs/                <- everything the run produces, nothing you type
|   |-- B1_diagrams.png
|   |-- B1_envelope.png
|   |-- B1_section.png
|   |-- B1_report.html
|   |-- B1_report.md

(.venv) $ tree ~/code/engineeringengine -L 2  # the LIBRARY -- a different folder

engineeringengine/
|-- pyproject.toml
|-- src/austruct/
|   |-- materials/          [1] reference data
|   |-- project/ loads/     [2] project data
|   |-- analysis/ structures/ [3] fast demand calculation
|   |-- design_documentation/ [4] design documentation
|   |-- design/ sections/   [5] design verification
|   |-- report/             [6] reporting
|-- tests/                  799 passing
```

**Screenshot 2.3** — the job folder (top) and the library folder (bottom) are separate, and only one of them is installed. `outputs/` holds only generated files, so you can delete it at any time and re-run to get it back — which is a good habit: if you cannot delete it, something in there was hand-edited.

#### A CHECKLIST YOU CAN RE-USE FOR EVERY NEW JOB

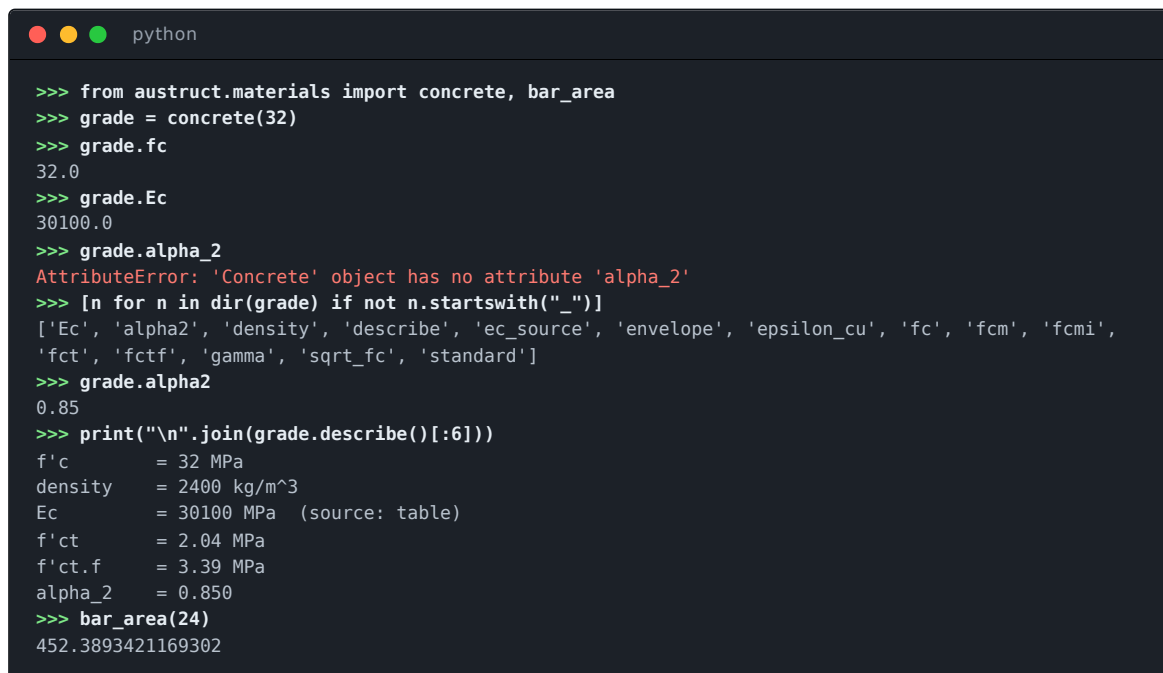
- 1 Copy the two-file job folder from your last job. Do not start from a blank page.
- 2 Open a terminal, activate the environment, and confirm `(.venv)` appears.
- 3 `cd` into `job/`. Everything runs from there.
- 4 Edit `job_data.py` first, completely, before touching the member file.
- 5 Run early and often. A five-line script that runs beats a fifty-line one that does not.

### 3 Finding out what the library can do

Before importing anything into a job file, get into the habit of poking at it interactively. Type `python` with no arguments and you get a prompt where every line runs as you press enter. Nothing you do there can break a file.

#### Reference data — ASET component 1

Start with the simplest thing in the toolkit: a concrete grade. It is looked up from a JSON file, not calculated, and one call gets you everything AS 3600 derives from  $f'_c$ .



```
python

>>> from austruct.materials import concrete, bar_area
>>> grade = concrete(32)
>>> grade.fc
32.0
>>> grade.Ec
30100.0
>>> grade.alpha_2
AttributeError: 'Concrete' object has no attribute 'alpha_2'
>>> [n for n in dir(grade) if not n.startswith("_")]
['Ec', 'alpha2', 'density', 'describe', 'ec_source', 'envelope', 'epsilon_cu', 'fc', 'fcm', 'fcmi',
'fct', 'fctf', 'gamma', 'sqrt_fc', 'standard']
>>> grade.alpha2
0.85
>>> print("\n".join(grade.describe()[6]))
f'c      = 32 MPa
density  = 2400 kg/m^3
Ec       = 30100 MPa (source: table)
f'ct     = 2.04 MPa
f'ct.f   = 3.39 MPa
alpha_2  = 0.850
>>> bar_area(24)
452.3893421169302
```

**Screenshot 3.1** — the third line is a deliberate mistake, left in. Guessing an attribute name is normal; the recovery is `dir()`, which lists everything the object actually has. That two-line loop is the most useful debugging habit in Python.

Two things in that panel are worth dwelling on. `Ec = 30100 MPa (source: table)` tells you the modulus came from Table 3.1.2 rather than the expression — the library records *which* route it took, not just the answer. And `bar_area(24)` returns `452.389...`, not a rounded 450: reference data is held exactly and rounded only when printed.

#### The same trick on a design module

`dir()` on a package tells you what checks exist. The docstring tells you what each one wants. Between them you rarely need to open the source.

```
python

>>> from astruct.design import as3600
>>> [n for n in dir(as3600) if not n.startswith("_")]
['CostRates', 'DevelopmentLength', 'OptimisationResult', 'SearchBounds', 'SectionCost', 'SizeRange',
'cd_dimension', 'check_bar_fit', 'check_crack_control', 'check_deflection', 'check_detailing',
'check_flexure', 'check_shear', 'check_span_to_depth', 'compression_development_length', 'constants',
'curtailment_extension', 'detailing', 'development_length', 'effective_shear_depth',
'effective_stiffness', 'flexure', 'ief_max', 'lap_length', 'minimum_clear_spacing',
'minimum_cost_section', 'minimum_shear_reinforcement', 'minimum_steel_area', 'moment_capacity',
'optimise', 'redistribution_limit', 'required_shear_reinforcement', 'required_steel_area',
'serviceability', 'shear', 'shear_capacity']
>>> print(as3600.check_shear.__doc__[:900])
Check a section's shear capacity against a design shear force.

    Basis
    ----
    As :func:`shear_capacity`, plus ``V* <= phi.V_u`` (Cl 2.2.2).

    Parameters
    -----
    V_star:
        Design shear force (N), magnitude. Take it from
        :meth:`~austruct.analysis.results.BeamResults.shear_at_d_from_support`
        where the support introduces compression, or from
        :attr:`~austruct.analysis.results.BeamResults.max_shear` otherwise.
    M_star:
        Coexisting design moment (N.mm). Used by the general method only.

    Returns
    -----
    CalcResult
```

**Screenshot 3.2** — every public calculation in AS 3600, listed by the library itself. The docstring of `check_shear` names its Basis (which clauses), its Parameters (what you must supply) and its Returns.

### THE THREE PLACES TO LOOK, IN ORDER

**1.** `dir(module)` in the REPL — what exists. **2.** `print(thing.__doc__)` or `help(thing)` — what it wants. **3.** `CATALOGUE.md` in the repository root — every registered module, its clause coverage and its verification status, generated from the code itself so it cannot go stale.

## Envelope versus check — the distinction that makes the library safe

This is the most important idea in the toolkit and it takes thirty seconds to see. Ask for 150 MPa concrete and the module *refuses*. Ask for 100 and it answers.

```
python

>>> from astruct.materials import concrete
>>> concrete(150)
austruct.core.exceptions.OutsideEnvelope: Inputs outside validity envelope [Concrete properties (AS
3600:2018)]: 20 <= f'c <= 100 MPa [actual 150]  OUTSIDE
>>> concrete(100)
Concrete(fc=100.0, density=2400.0, standard=Standard(code='AS 3600', edition=2018, title='Concrete
structures', amendments=()), fcmi=99.0, fcm=110.0, Ec=42200.0, ec_source='table')
```

**Screenshot 3.3** — an `OutsideEnvelope` exception, not a number. The message names the limit, the clause behind it, and your value.

|                    | Envelope                               | Check                                   |
|--------------------|----------------------------------------|-----------------------------------------|
| What it bounds     | The range in which the method is valid | A code criterion                        |
| Breaching it means | The answer is <i>unknown</i>           | The answer is <i>known and it fails</i> |
| What you get       | An exception. No number at all.        | A valid result that reads FAIL          |
| Example            | $f'_c = 150 \text{ MPa}$               | $M^* > \phi M_{u0}$                     |

An overstressed beam is a perfectly good calculation with a failing result — you want to see that, with all the working, so you can size up. A beam of 150 MPa concrete is not a calculation this library can do at all, and quietly extrapolating would be the worst possible outcome. Spreadsheets almost never make this distinction. It is worth insisting on it in anything you build.

## 4 File one — the job (ASET component 2)

Write this file completely before you touch the member file. It contains no capacities and no Standards arithmetic beyond turning site data into loads. If you later design B2, B3 and the transfer slab, they all import this same file — and they all move together when the client changes the plant weight.

### The job record

`Project` is a plain record of client-supplied facts. It looks like bookkeeping, and then it quietly does real work: the occupancy you state drives the  $\psi$  combination factors, and the structure type decides whether you get the AS/NZS 1170.0 building combinations or the AS 5100.2 bridge set. Stating "office building" once is what stops a bridge combination reaching a floor beam.

```
job/job_data.py
23  # 1. The job record -- client-supplied facts, written down once
24  # -----
25  PROJECT = Project(
26      job_number="25-0142",
27      job_name="Northbank Depot",
28      client="Northbank Logistics Pty Ltd",
29      site_address="14 Kembla Street, Wollongong NSW",
30      jurisdiction="NSW",
31      structure_type=StructureType.BUILDING,
32      occupancy=Occupancy.OFFICE,
33      exposure=ExposureClassification.B1,
34      design_life_years=50,
35      importance_level=2,
36      engineer="A. Morrison",
37      checker="",
38  )
39
40  # -----
41
```

**Screenshot 4.1** — note what is *not* here: no defaults invented on your behalf. An unstated design life stays empty so the report can say 'not stated' rather than printing a number nobody chose.

### Geometry and areal loads

Spans and tributary widths get names, so that every later use of them is self-evidently the same number. The library works in newtons and millimetres — which makes 1 kN/m exactly 1 N/mm — so areal loads are written in kPa the way a load schedule states them and converted exactly once, here, by multiplying by `kPa`.

job/job\_data.py

```
41 # 2. Geometry the loads depend on
42 # -----
43 SPAN = 7.2 * m
44 """Centre-to-centre of the supporting blockwork corbels."""
45
46 TRIB_WIDTH = 3.6 * m
47 """Half the bay each side of B1 -- one-way slab spanning onto it."""
48
49 PLANT_POSITION = 3.0 * m
50 """Chainage of the condenser unit's nearer support foot, from grid A."""
51
52 # -----
53 # 3. Areal loads -> line loads on B1
54 # -----
55 # [ASSUMPTION] 180 mm one-way slab, 24 kN/m3 reinforced concrete.
56 G_AREAL = 4.32 * kPa + 0.90 * kPa
57 """Permanent action on the roof: 180 slab (4.32 kPa) + membrane, insulation and
58 services (0.90 kPa). Does NOT include beam self weight -- the library derives
59 that from the section, see `SelfWeight` in beam_B1.py."""
60
61 Q_AREAL = 0.25 * kPa
62 """Imposed action, non-trafficable roof, AS/NZS 1170.1 Table 3.2."""
63
64 Q_PLANT = 40.0 * kN
65 """Rooftop condenser, operating weight, supplier data sheet NB-CD-04 Rev B."""
66
67
68 # -----
69
```

**Screenshot 4.2** — the triple-quoted string under each value is a docstring, and it is doing the job a cell comment does in a spreadsheet, except it cannot be scrolled out of view or lost in a copy-paste.

#### UNITS: READ THIS ONCE AND STOP WORRYING

Everything is N, mm and MPa. The constants `kN`, `m`, `kPa`, `kNm` and `kN_per_m` are plain multipliers that convert *into* that system on the way in (`7.2 * m` is 7200.0) and *out of* it on the way to a print statement (`M / kNm`). If a number ever looks wrong by a factor of 1000, this is where to look first.

### Wind, to AS/NZS 1170.2

The library does not derive site wind — that is a job-specific piece of work with too many local judgements to hide inside a package. So it goes in your file, as a small function with each multiplier on its own line, tagged with the clause it comes from. This is exactly the pattern to copy whenever you need a calculation the library does not have.

job/job\_data.py

```
69 # 4. Wind, to AS/NZS 1170.2
70 # -----
71 # [VECTOR] Every factor below is transcribed from the standard for THIS site
72 # and needs checking against the printed clause before issue.
73 V_R = 45.0
74 """Regional wind speed V_500, Region A4, AS/NZS 1170.2 Table 3.1 (m/s)."""
75
76 M_D = 1.00 # wind direction multiplier, Cl 3.3
77 M_ZCAT = 0.91 # terrain/height multiplier, TC2, z = 9.0 m, Table 4.1
78 M_S = 1.00 # shielding, Cl 4.3
79 M_T = 1.00 # topographic, Cl 4.4
80
81 C_PE = -0.90 # external pressure coefficient, roof, Table 5.3(A)
82 C_PI = 0.20 # internal pressure coefficient, Table 5.1(A)
83 K_A = 1.00 # area reduction, Cl 5.4.2
84 K_L = 1.00 # local pressure, Cl 5.4.4
85 C_DYN = 1.00 # dynamic response, Cl 6.1
86
87 AIR_DENSITY = 1.2e-6
88 """1.2 kg/m3 expressed in the library's N-mm system, so that
89 0.5 * rho * V^2 comes out in MPa (= N/mm2 = kPa/1000)."""
90
91
92 def design_wind_speed() -> float:
93     """V_des, theta to AS/NZS 1170.2 Cl 2.3, in m/s.
94
95     [BASIS] AS/NZS 1170.2 Cl 2.3: V_sit = V_R * M_d * (M_z,cat M_s M_t)
96     """
97     return V_R * M_D * (M_ZCAT * M_S * M_T)
98
99
100 def net_uplift_pressure() -> float:
101     """Net upward pressure on the roof, in the library's units (MPa).
102
103     [BASIS] AS/NZS 1170.2 Cl 2.4.1: p = 0.5 rho_air V_des^2 C_fig C_dyn
104     with C_fig = C_p,e K_a K_c K_l K_p - C_p,i (Cl 5.2)
105
106     Returned POSITIVE for uplift; the sign convention is applied at the call
107     site in beam_B1.py, where the load direction is visible next to the beam.
108     """
109     v_des = design_wind_speed()
110     c_fig = (C_PE * K_A * K_L) - C_PI # negative = suction/uplift
111     p = 0.5 * AIR_DENSITY * v_des**2 * c_fig * C_DYN
112     return abs(p)
113
114
115 # -----
116
```

**Screenshot 4.3** — one factor per line, each with its clause. When the checker asks ‘where did  $M_{z,cat} = 0.91$  come from?’, the answer is on the same line as the number.

Two habits in that panel are worth stealing. The `[VECTOR]` tag marks values transcribed from a Standard, so a reviewer can find every one of them with a text search. And `net_uplift_pressure()` returns a magnitude only, with the docstring saying the sign is applied at the call site — because the direction of a load is easier to get right where you can see the beam it is on.

## Derived line loads — the file's actual output

job/job\_data.py

```
116 # 5. Derived line loads -- what beam_B1.py actually consumes
117 # -----
118 W_G = G_AREAL * TRIB_WIDTH
119 """Permanent UDL on B1, N/mm (= kN/m)."""
120
121 W_Q = Q_AREAL * TRIB_WIDTH
122 """Imposed UDL on B1, N/mm."""
123
124 W_WU = net_uplift_pressure() * TRIB_WIDTH
125 """Ultimate wind UDL on B1, N/mm, magnitude only (uplift)."""
126
```

**Screenshot 4.4** — three numbers, in the library's units, ready to be consumed. Everything above exists to produce these.

## 5 File two — the member

Now the calculation itself. It opens with a block of imports, and that block is worth reading slowly, because it is the answer to the question this tutorial exists for: *how do I call vetted modules into a different file?*

```
job/beam_B1.py
15 from pathlib import Path
16
17 from austruct.analysis import UDL, PointLoad, SelfWeight, analyse_combinations, simply_supported
18 from austruct.core.units import kN, kN_per_m, kNm, kPa, m
19 from austruct.design import as3600
20 from austruct.design_documentation import designate, parse
21 from austruct.loads import ActionType, LoadCase
22 from austruct.materials import describe_options
23 from austruct.report import HtmlRenderer, MarkdownRenderer, Report, plots
24
25 # Your own job file, sitting next to this one. Same import machinery, no
26 # package install -- Python looks in the script's own directory first.
27 import job_data as job
28
```

**Screenshot 5.1** — six imports from the installed library, then one from the file next door. The blank line between them is a convention worth keeping: above it, code you did not write; below it, code you did.

Read down the imports and you can predict the whole calculation before reading a line of it: something about beams and loads, units, AS 3600, a designation parser, load cases, a bar catalogue, a report. Imports at the top of a file are a table of contents.

### The design decision, as one line of text (ASET component 4)

Here is where the toolkit does something a spreadsheet cannot. The beam is described by a string in a defined grammar — the same string that goes on the drawing — and the library turns it into a section object.

```
job/beam_B1.py
30 # 1. The design decision, written the way it goes on the drawing
31 # -----
32 DESIGNATION = "350 x 650 | C32 | COV 40 | BOT 3-N24 | LIG N12-2L@250"
33 section = parse(DSIGNATION, name="B1")
34
35 assert designate(section) == DESIGNATION, "designation does not round-trip"
36
37 print("SECTION")
38 for line in section.describe():
39     print("    ", line)
40
41 # -----
42
```

**Screenshot 5.2** — `parse()` in, `designate()` back out. The `assert` on line 34 is a one-line guard that the round trip is exact.

| Field          | Means                        | Rules                                                                                                                                                                                                        |
|----------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 350 x 650      | $b \times D$ , mm            | Always first                                                                                                                                                                                                 |
| C32            | $f'_c = 32$ MPa              | All keyword-led and order-independent. An unrecognised field <b>raises</b> rather than being ignored — silently dropping TOP 2-N16 would hand you a singly reinforced capacity for a doubly reinforced beam. |
| COV 40         | Cover to the fitment, mm     |                                                                                                                                                                                                              |
| BOT 3-N24      | Three N24 bottom bars        |                                                                                                                                                                                                              |
| LIG N12-2L@250 | N12 two-leg ligatures at 250 |                                                                                                                                                                                                              |

### WHY A STRING, AND NOT FIVE ARGUMENTS

Because that string is the thing everyone else in the project already reads. It goes in the schedule, on the drawing, in the email to the drafter, and into `parse()` — unchanged. There is no transcription step between the design decision and the calculation, so there is no transcription error. And because `designate(parse(s)) == s` is asserted, the calculation cannot quietly be for a different beam than the one you wrote down.

```
python beam_B1.py — first block

SECTION
    Section      = B1
    D            = 650 mm
    A_g          = 227500 mm^2
    y_c          = 325.0 mm from top
    I_g          = 8.01e+09 mm^4
    Z_bottom     = 2.465e+07 mm^3
    b_v          = 350 mm

    Concrete:
        f'c       = 32 MPa
        density   = 2400 kg/m^3
        Ec        = 30100 MPa (source: table)
        f'ct      = 2.04 MPa
        f'ct.f    = 3.39 MPa
        alpha_2   = 0.850
        gamma     = 0.826
        epsilon_cu = 0.0030

    Longitudinal reinforcement:
        3-N24 at d = 586 mm (bottom main)

    Fitments:    N12-2leg @ 250
```

**Screenshot 5.3** — what that one line expanded into. Note  $d = 586$  mm, derived from cover + fitment + half a bar, not typed. That subtraction is one of the most common arithmetic slips in hand calculations, and here it happens once, in tested code.

## 6 Load cases and combinations (AS/NZS 1170)

A load case says *what the loads are and which action they represent*. A combination says *how the jurisdiction requires them to be added up*. Keeping those separate is why you never write `1.2 * dead + 1.5 * live` by hand again.

### The cases — unfactored, and tagged

```
job/beam_B1.py
42 # 2. Load cases -- unfactored, each tagged with the action it represents
43 # -----
44 cases = (
45     LoadCase(
46         "G_roof",
47         ActionType.G,
48         (
49             UDL(magnitude=job.W_G),
50             SelfWeight(area=section.geometry.area, density=2400.0, length=job.SPAN),
51         ),
52     ),
53     LoadCase(
54         "Q_roof",
55         ActionType.Q,
56         (
57             UDL(magnitude=job.W_Q),
58             PointLoad(position=job.PLANT_POSITION, magnitude=job.Q_PLANT),
59         ),
60     ),
61     LoadCase(
62         "Wu_uplift",
63         ActionType.Wu,
64         # [UNITS] Upward is negative in this package's sign convention.
65         (UDL(magnitude=-job.W_WU),),
66     ),
67 )
68
```

**Screenshot 6.1** — three cases. Each carries an `ActionType`, and that tag is the whole mechanism: it is what connects ‘this 18.79 kN/m is permanent action’ to ‘the code multiplies permanent action by 1.2 in this combination’.

Four things in that panel repay attention:

- **SelfWeight instead of a number.** It derives the UDL from the section's own area and a density, so the moment you change  $350 \times 650$  to  $400 \times 750$  the self weight follows. A hand-computed 5.36 would not.
- **A UDL and a point load in the same case.** A load case is a group of loads, not one load: the roof imposed pressure and the plant unit are both Q and must be factored together.
- **The minus sign on the wind UDL.** Downward is positive throughout, so uplift is negative. It is applied here, next to the beam, rather than buried in `job_data.py`.
- **Names you will read again.** `"G_roof"` and `"Wu_uplift"` come back out in the envelope as the governing case, so make them say something.

```
python beam_B1.py

LOAD CASES (unfactored)
  G_roof    G    UDL      18.79 kN/m
  G_roof    G    SelfWeight  5.36 kN/m
  Q_roof    Q    UDL      0.90 kN/m
  Q_roof    Q    PointLoad  40.0 kN at 3.0 m
  Wu_uplift Wu    UDL     -3.98 kN/m
```

**Screenshot 6.2** — every load echoed back with its type and intensity. Reading this block before looking at any capacity is the equivalent of checking your load take-down before you start designing.

## The combinations come from the project record, not from you

You never type a load factor. `PROJECT.load_combinations(sls=False)` reads the structure type and the occupancy off the record you wrote in part 4 and returns the AS/NZS 1170.0 Cl 4.2.2 ultimate set, with the  $\psi$  factors already resolved for an office.

```
python

>>> from astruct.project import Occupancy, Project, StructureType
>>> p = Project(job_number="25-0142", occupancy=Occupancy.OFFICE,
...           structure_type=StructureType.BUILDING, engineer="A. Morrison")
>>> p.combination_factors()
(0.4, 0.7, 0.4)
>>> for c in p.load_combinations(sls=False):
...     print(" ", c)
  ULS1: 1.35G
  ULS2: 1.2G + 1.5Q
  ULS3: 1.2G + 0.6Q (long term)
  ULS4: 1.2G + Wu + 0.4Q
  ULS5: 0.9G + Wu (uplift/reversal)
  ULS6: G + Eu + 0.4Q
  ULS7: 1.2G + Su + 0.4Q
```

**Screenshot 6.3** — `(0.4, 0.7, 0.4)` is  $(\psi_c, \psi_s, \psi_i)$  for an office, and you can see them arrive in ULS3 and ULS4. Change `Occupancy.OFFICE` to `STORAGE` and every combination below changes with it.

### COMBINATIONS YOU DID NOT ASK FOR ARE A FEATURE

Seven come back, including earthquake and snow, because the *project* implies them. The analysis step drops the ones no supplied case contributes to, so ULS6 and ULS7 cost nothing here. The point is that the list is generated from the jurisdiction, not from what you remembered on the day — and the one you forget is the one that governs.

```
python beam_B1.py

COMBINATIONS (AS/NZS 1170.0 Cl 4.2.2, psi factors from the occupancy)
  ULS1: 1.35G
  ULS2: 1.2G + 1.5Q
  ULS3: 1.2G + 0.6Q (long term)
  ULS4: 1.2G + Wu + 0.4Q
  ULS5: 0.9G + Wu (uplift/reversal)
  ULS6: G + Eu + 0.4Q
  ULS7: 1.2G + Su + 0.4Q
```

**Screenshot 6.4** — the same seven, echoed by the job script. ULS5 ( $0.9G + W_u$ ) is the uplift/reversal case: with  $0.9G$  at  $21.7 \text{ kN/m}$  against  $3.98 \text{ kN/m}$  of uplift there is no reversal here, but the combination is run and the result recorded rather than assumed.

## 7 Analyse, then envelope (ASET component 3)

Two lines of code do the whole of the analysis: build the member, then run every combination over it and keep the worst of each action — along with a record of which combination caused it.

```
job/beam_B1.py
82 # 3. Analyse over every ULS combination the project record implies
83 # -----
84 combinations = job.PROJECT.load_combinations(sls=False)
85
86 print("\nCOMBINATIONS (AS/NZS 1170.0 Cl 4.2.2, psi factors from the occupancy)")
87 for combo in combinations:
88     print("    ", combo)
89
90 beam = simply_supported(job.SPAN, section=section, name="B1")
91 env = analyse_combinations(bean, cases, combinations)
92
93 print("\nENVELOPE")
94 print(env.summary())
95
96 # -----
97
```

**Screenshot 7.1** — `simply_supported()` is one of a family ( `cantilever` , `propped` , `continuous` ). Passing `section=` rather than a bare EI means the deflections use the section you actually detailed.

```
python beam_B1.py

ENVELOPE
Envelope: B1 [ULS]
7 combination(s) analysed

ULS1: 1.35G
ULS2: 1.2G + 1.5Q
ULS3: 1.2G + 0.6Q (long term)
ULS4: 1.2G + Wu + 0.4Q
ULS5: 0.9G + Wu (uplift/reversal)
ULS6: G + Eu + 0.4Q
ULS7: 1.2G + Su + 0.4Q

Governing actions:
M_max (sagging)      296.07 kN.m at x = 3.000 m [ULS2]
M_min (hogging)      0.00 kN.m at x = 0.000 m [ULS1]
V_max               144.18 kN at x = 0.000 m [ULS2]
V_min              -134.18 kN at x = 7.200 m [ULS2]
Deflection max       6.26 mm at x = 3.538 m [ULS2]

Reactions (positive upward):
A      max    144.18 kN [ULS2]   min    63.90 kN [ULS5]
B      max    134.18 kN [ULS2]   min    63.90 kN [ULS5]
```

**Screenshot 7.2** — the governing action table. Every row carries the combination that caused it in square brackets, which is the single most useful thing an envelope can tell you.

Read that block the way you would read the summary page of a frame analysis:

- **M\* = 296.1 kN.m at x = 3.000 m [ULS2]** — sagging, under the plant load, from 1.2G + 1.5Q. Not at midspan, because the point load is not at midspan.
- **M<sub>hog</sub> = 0.00 kN.m** — no reversal anywhere, under any of the seven. The uplift case never overcomes 0.9G.

- **Reactions, max and min.** The minimum at each support is 69.43 kN from ULS5, still comfortably downward: no hold-down required. That is a real check, and it fell out of the envelope for free.
- **Deflection 6.26 mm [ULS2]** — from *factored* loads on a gross section, so it is a solver output, not a serviceability check. Do that separately, with SLS combinations and `as3600.effective_stiffness`.

#### ONE MESH, SHARED BY EVERY COMBINATION

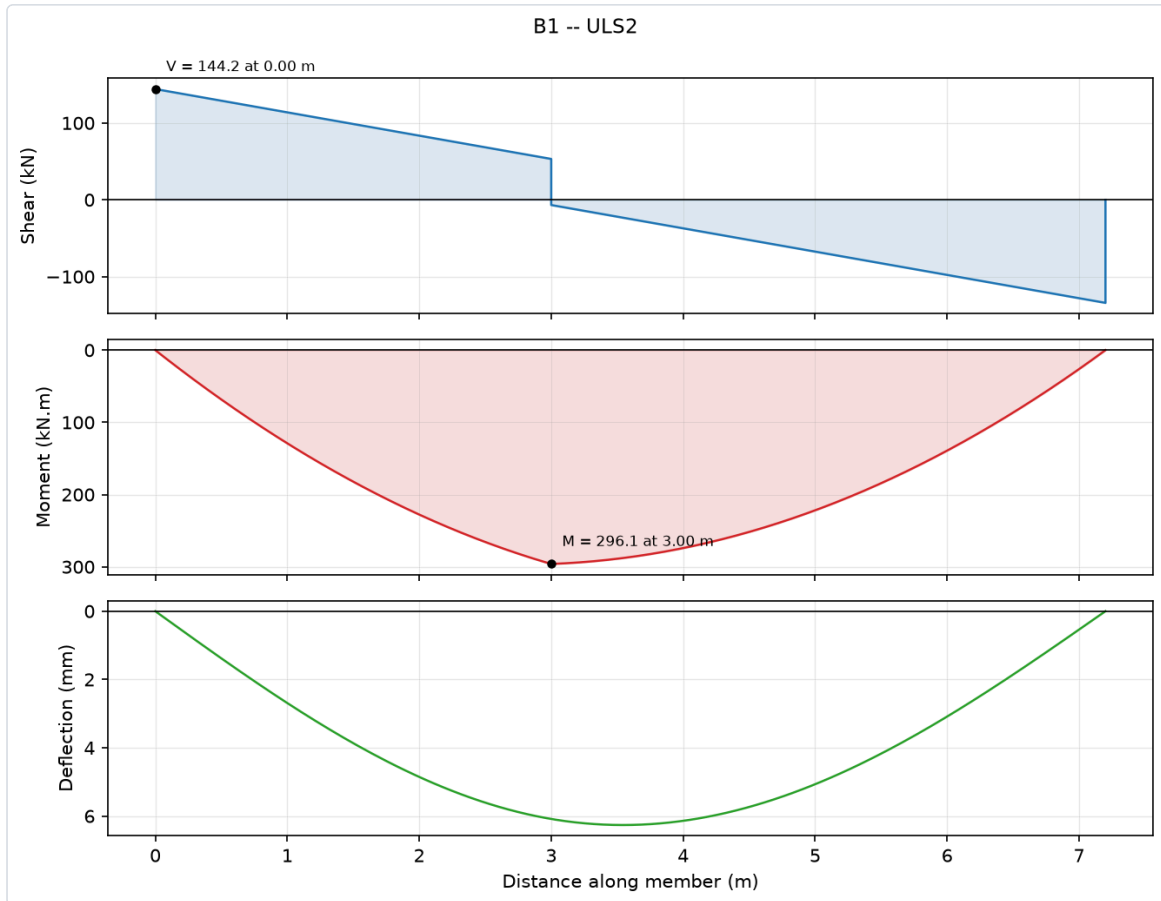
Meshed independently, each combination would land on a slightly different set of x-positions, and enveloping them would need interpolation — which smears exactly the shear discontinuities an envelope exists to capture. The library gives every combination the union of all mesh points, so the envelope is element-wise and exact. You get the sharp step under the plant load, not a rounded corner.

## Seeing it

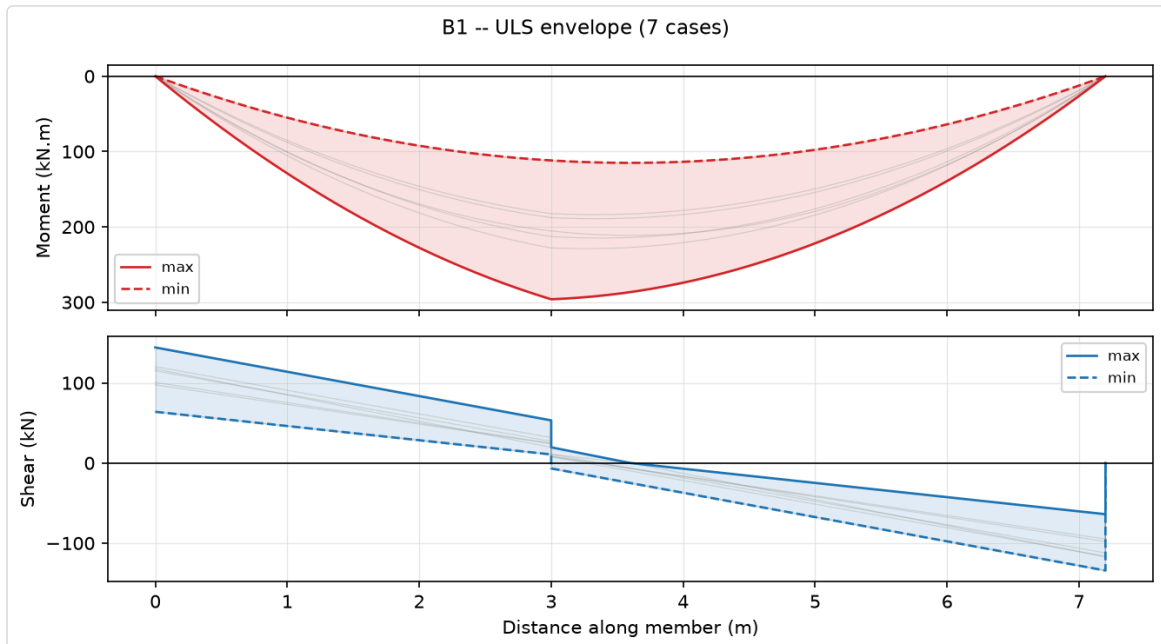
Two plots, both one line of code. The first is the governing combination alone, drawn the way you would draw it by hand; the second is the envelope with all seven cases faint behind it.

```
job/beam_B1.py
129 # The governing combination on its own, so the shear, moment and deflection
130 # diagrams can be read the way they are drawn by hand. `combo.apply(cases)`
131 # is the same factoring the envelope did -- done once here, visibly.
132 governing_combo = next(c for c in combinations if c.name == env.moment.governing_combo)
133 governing_beam = simply_supported(
134     job.SPAN, loads=governing_combo.apply(cases), section=section, name=f"B1 --
    {governing_combo.name}"
135 )
136 fig_dia = plots.plot_diagrams(governing_beam.solve())
137 plots.save(fig_dia, f"{OUT}/B1_diagrams.png")
138
139 fig_env = plots.plot_envelope(env, show_cases=True)
140 plots.save(fig_env, f"{OUT}/B1_envelope.png")
141
```

**Screenshot 7.3** — `combo.apply(cases)` is the same factoring the envelope did internally, done once here where you can see it. That is the layering working: nothing is hidden, it is just not repeated.



**Screenshot 7.4** — ULS2. Sagging is drawn downward, the engineering convention, so the moment diagram hangs the way the beam does. The shear step at 3.0 m is the 60 kN factored plant load;  $V = 144.2$  kN at the support, reduced to 126.4 kN at d for the shear check.



**Screenshot 7.5** — the envelope, with every contributing combination faint behind it. The dashed *min* curve is the lower bound: driven by ULS5 ( $0.9G + W_{Uj}$ ), it still sits at about 90 kN.m of sagging at midspan, which is the picture of 'no reversal anywhere'. Seeing that is worth more than reading  $M_{hog} = 0$ .

## 8 Flexure to AS 3600:2018

After all that, the design itself is three lines. That is the correct proportion: the hard part of a calculation is getting the right demand to the right section, not evaluating the capacity once you have.

```
job/beam_B1.py
97 # 4. Design verification to AS 3600:2018
98 # -----
99 M_star = env.M_star
100 V_star = env.shear_at_d_from_support(section.d)
101
102 flexure = as3600.check_flexure(section, M_star)
103 shear = as3600.check_shear(section, V_star=V_star, M_star=M_star)
104
```

**Screenshot 8.1** — `shear_at_d_from_support(d)` is the code allowance for taking the design shear at d from a support that introduces compression. It is a method on the envelope because only the envelope knows the shear diagram.

```
python beam_B1.py

AS 3600:2018 CHECKS
M* = 296.1 kN.m [ULS2] phi.Muo = 317.4 kN.m util 0.933 PASS
V* = 126.4 kN [ULS2] phi.Vu = 339.5 kN util 0.372 PASS

Reversal check: M*_hogging = 0.0 kN.m -> no reversal, section as detailed
```

**Screenshot 8.2** — both checks, with the governing combination, the capacity, the utilisation and a verdict. Flexure at 0.933 is a sensibly worked section; shear at 0.372 is not, and part 10 comes back to that.

### What is actually inside `check_flexure`

The function returns a `CalcResult`, never a bare float. That object carries the inputs it was given, the clauses it applied, the validity envelope it enforced, the intermediate working, the results, and every pass/fail criterion — which is what makes the report in part 11 possible without you assembling anything.

```
python

>>> from austruct.core.units import kNm
>>> from austruct.design import as3600
>>> from austruct.design_documentation import parse
>>> section = parse("350 x 650 | C32 | COV 40 | BOT 3-N24 | LIG N12-2L@250")
>>> flexure = as3600.check_flexure(section, 296.1 * kNm)
>>> for ref in flexure.basis:
...     print(" ", ref)
AS 3600:2018 Cl 8.1.2 -- Ultimate strength in bending
AS 3600:2018 Cl 8.1.3 -- Rectangular stress block
AS 3600:2018 Table 2.2.2 -- Capacity reduction factors
AS 3600:2018 Cl 8.1.5 -- Ductility limit on k_uo
AS 3600:2018 Cl 8.1.6.1 -- Minimum strength requirement
>>> for c in flexure.checks:
...     print(f" {c.label:<34} {c.actual / c.display_factor:9.2f} {c.operator} "
...           f"{c.limit / c.display_factor:9.2f} {c.display_unit or c.unit}")
Ductility, k_uo          0.15 <=      0.36 -
Minimum strength, M_uo >= 1.2 M_cr  373.47 >=    109.57 kN.m
Flexural strength, M* <= phi.M_uo  296.10 <=    317.45 kN.m
>>> for lim in flexure.envelope.limits:
...     print(f" {lim.name:<10} {lim.value:8.0f} {lim.unit:<8} allowed {lim.lower:g} to
{lim.upper:g}")
f'c          32 MPa      allowed 20 to 100
f'c          32 MPa      allowed 20 to 100
density      2400 kg/m^3  allowed 1800 to 2800
>>> for note in flexure.envelope.notes:
...     print(" -", note)
- Non-prestressed reinforced concrete only. No axial force.
- Sagging bending with the compression face at the top of the section as supplied. Model hogging
  by inverting the section.
- Torsion, lateral instability of slender beams and the effects of sustained load are not
  considered.
>>> flexure.provenance.module
'austruct.design.as3600.flexure'
>>> flexure.provenance.status.value
'UNVERIFIED'
```

**Screenshot 8.3** — the audit trail, interrogated from the REPL. Five clauses in the order they were applied; three checks, not one; the envelope that was enforced; and the module's own verification status.

Notice that there are **three** flexural checks, not just  $M^* \leq \phi M_{uo}$ :

| Check                         | Clause     | Here                           | Why it exists                                                                                                                |
|-------------------------------|------------|--------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Ductility, $k_{uo} \leq 0.36$ | Cl 8.1.5   | 0.147 — PASS                   | A section that crushes before the steel yields fails without warning. This is the check a hand calculation most often skips. |
| $M_{uo} \geq 1.2 M_{cr}$      | Cl 8.1.6.1 | $373 \geq 110$ kN.m — PASS     | Minimum strength: the beam must not fail the instant it cracks.                                                              |
| $M^* \leq \phi M_{uo}$        | Cl 2.2.2   | $296.1 \leq 317.4$ kN.m — PASS | The strength check everyone remembers.                                                                                       |

#### READ THE ENVELOPE NOTES, EVERY TIME

The three notes in Screenshot 8.3 are the module telling you what it did *not* do: no axial force, no torsion, no lateral instability of slender beams, and *sagging only* — "model hogging by inverting the section". That last one is a real trap on a continuous beam. The library will not warn you again; it has said it here, in the result you already have in your hand.

## 9 Shear to AS 3600:2018

Same call shape, one extra argument — and that extra argument is the interesting part.

`as3600.check_shear(section, V_star=V_star, M_star=M_star)` takes the coexisting moment as well as the shear. That is not padding. AS 3600:2018 moved to a modified compression field theory formulation in which the concrete contribution depends on the longitudinal strain in the web, which depends on  $M^*$ . Shear capacity is *coupled to the demand*. Ferster's component 5 calls this out specifically as the case that makes a verification tool hard to build, and it is why you want the coupling handled inside a tested module rather than by remembering to look up a second table.

| outputs/B1_report.html                                                                        |                                                         |        |                     |                     |             |        |
|-----------------------------------------------------------------------------------------------|---------------------------------------------------------|--------|---------------------|---------------------|-------------|--------|
| WORKING                                                                                       |                                                         |        |                     |                     |             |        |
| Symbol                                                                                        | Description                                             | Value  | Unit                |                     |             |        |
| d_v                                                                                           | Effective shear depth                                   | 527.4  | mm                  |                     |             |        |
| d_o                                                                                           | Depth to outermost tensile bar                          | 586    | mm                  |                     |             |        |
| (A_sv/s)_min                                                                                  | Minimum shear reinforcement                             | 0.3168 | mm <sup>2</sup> /mm |                     |             |        |
| k_v                                                                                           | Concrete shear factor (simplified, >= minimum fitments) | 0.15   | -                   |                     |             |        |
| theta_v                                                                                       | Compression strut angle                                 | 36     | deg                 |                     |             |        |
| V_uc + V_us                                                                                   | Before the V_u,max cap                                  | 485    | kN                  |                     |             |        |
| RESULTS                                                                                       |                                                         |        |                     |                     |             |        |
| Symbol                                                                                        | Description                                             | Value  | Unit                |                     |             |        |
| V_uc                                                                                          | Concrete contribution                                   | 156.6  | kN                  |                     |             |        |
| V_us                                                                                          | Fitment contribution                                    | 328.4  | kN                  |                     |             |        |
| V_u,max                                                                                       | Web crushing limit                                      | 1545   | kN                  |                     |             |        |
| V_u                                                                                           | Nominal shear capacity                                  | 485    | kN                  |                     |             |        |
| phi                                                                                           | Capacity reduction factor, shear                        | 0.7    | -                   |                     |             |        |
| phi.V_u                                                                                       | Design shear capacity                                   | 339.5  | kN                  |                     |             |        |
| CHECKS                                                                                        |                                                         |        |                     |                     |             |        |
| Criterion                                                                                     | Actual                                                  |        | Limit               | Unit                | Utilisation | Status |
| Minimum shear reinforcement, A_sv/s                                                           | 0.9048                                                  | >=     | 0.3168              | mm <sup>2</sup> /mm | 0.350       | PASS   |
| Shear strength, V* <= phi.V_u                                                                 | 126.4                                                   | <=     | 339.5               | kN                  | 0.372       | PASS   |
| astruct.design.as3600.shear v0.1.0 · UNVERIFIED · not checked · run 2026-09-03T18:41:37+00:00 |                                                         |        |                     |                     |             |        |
| R1                                                                                            |                                                         |        |                     |                     |             |        |

**Screenshot 9.1** — the shear check as it appears in the generated report: inputs, seven clauses, the validity envelope, the working, the results and the checks.  $d_v = 527.4$  mm,  $k_v = 0.15$  and  $\theta_v = 36^\circ$  are the simplified-method values of CI 8.2.4.2, reported rather than assumed.

### The two checks, and the one that is easy to forget

| Check                                   | Clause             | Here                                                    |
|-----------------------------------------|--------------------|---------------------------------------------------------|
| Minimum shear reinforcement, $A_{sv}/s$ | CI 8.2.1.7         | $0.905 \geq 0.317 \text{ mm}^2/\text{mm}$ — PASS        |
| $V^* \leq \phi V_u$                     | CI 2.2.2 / 8.2.1.1 | $126.4 \leq 339.5 \text{ kN}$ — PASS, utilisation 0.372 |

The minimum-reinforcement check is not decoration: the simplified method's  $k_v = 0.15$  is only available *because* the section carries at least minimum fitments. Widen the ligatures past that limit and both the check and the concrete contribution change together. The report says so on the  $k_v$  row: "simplified,  $\geq$  minimum fitments".

#### AS 5100.5 IS NOT 'AS 3600 WITH DIFFERENT NUMBERS';

The library also implements AS 5100.5, and for flexure the two produce an identical nominal  $M_{u0}$  and differ only in  $\phi$ . For **shear** they share no code at all: AS 5100.5:2017 predates the 2018 revision and uses the  $\beta_1\beta_2\beta_3 / f_{cv} / d_o$  family. Swapping `as3600` for `as5100_5` in a shear line and expecting a small change is the most likely way to get a bridge wrong with this toolkit.

#### Reading a utilisation of 0.37

The shear result is a genuine engineering signal, not a rounding artefact: N12 two-leg ligatures at 250 are carrying nearly three times what this beam asks of them. The honest response is to widen the spacing until either the  $V^*$  check or the minimum-reinforcement check starts to bind, and let the two checks tell you where the limit is. That iteration is the subject of the next part.

## 10 The loop: FAIL is a result, not an error

The section in this tutorial did not arrive fully formed. It was the second try. Here is the first, and the thirty seconds it took to fix — which is the part of the workflow that actually saves you time.

```
python
>>> from austruct.core.units import kNm
>>> from austruct.design import as3600
>>> from austruct.design_documentation import parse
>>> M_star = 296.1 * kNm
>>> trial = parse("350 x 650 | C32 | COV 40 | BOT 3-N20 | LIG N12-2L@250")
>>> first = as3600.check_flexure(trial, M_star)
>>> print(f"{first.utilisation:.3f} {'PASS' if first.passed else 'FAIL'}")
1.312 FAIL
>>> as3600.required_steel_area(trial, M_star).get("Ast_req")
1255.2780456542969
>>> trial.Ast
942.4777960769379
>>> better = parse("350 x 650 | C32 | COV 40 | BOT 3-N24 | LIG N12-2L@250")
>>> second = as3600.check_flexure(better, M_star)
>>> print(f"{second.utilisation:.3f} {'PASS' if second.passed else 'FAIL'}")
0.933 PASS
```

**Screenshot 10.1** — 3-N20 gives 1.312, FAIL. `required_steel_area` says the section needs 1255 mm<sup>2</sup> and has 942. 3-N24 gives 1357 mm<sup>2</sup> and 0.933, PASS. No spreadsheet was reopened and no clause was re-read.

Three things make that loop fast, and all three are design choices you can copy into anything you build:

- **A failing check still returns a full result.** You get the utilisation, the working and the shortfall — not an exception. Compare that with the envelope refusal in Screenshot 3.3: different situation, different behaviour, deliberately.
- **The inverse problem is a first-class function.** `required_steel_area` answers "how much steel does this moment need?" directly, so you are not bisecting by hand.
- **The decision is one string.** Changing `BOT 3-N20` to `BOT 3-N24` is the entire edit. Nothing downstream needs touching: *d*, the self weight, the diagrams and the report all follow.

```
python beam_B1.py

Bar arrangements that would satisfy the required area:
Required As = 1260 mm^2

Arrangement      As prov      Excess
-----
12-N12            1357         7.7%
3-N24             1357         7.7%
7-N16             1407        11.7%
5-N20             1571        24.7%
2-N32             1608        27.7%
```

**Screenshot 10.2** — and if you want the options laid out, `describe_options` takes the required area and returns every practical arrangement with its excess. 3-N24 at 7.7% over is the obvious pick here; 12-N12 provides the same area and would not fit.

### WHEN TO STOP ITERATING BY HAND

Once you are running the same edit-and-check loop more than a few times, the library will do it for you: `as3600.minimum_cost_section` searches a grid of widths, depths and bar arrangements against concrete and steel rates and returns the cheapest section that passes. Same checks, same clauses — just driven by a search instead of by you. That is the natural next step after this tutorial, and it only becomes possible because the check was written as a function in the first place.

## The other two failure modes you will hit

Neither is a mistake in the code you wrote; both are the environment telling you where it is looking. Together with Screenshots 1.1 and 1.2 they cover most of a first week.

```
bash

(.venv) $ cd ~/jobs/25-0142-northbank          # one level above the script
(.venv) $ python job/beam_B1.py                 # with a bare OUT = "../outputs"
...analysis and design output, as normal...
Traceback (most recent call last):
  File "./job/beam_B1.py", line 137, in <module>
    plots.save(fig_dia, f"{OUT}/B1_diagrams.png")
    ... 9 frames inside matplotlib and PIL omitted ...
FileNotFoundError: [Errno 2] No such file or directory: '../outputs/B1_diagrams.png'
```

**Screenshot 10.3** — the shipped file with its output line replaced by a bare `OUT = "../outputs"`, run from one directory up. The imports all resolve — Python adds the *script's* folder to the path, not yours — but the relative output path does not. The fix is the line the file actually ships with: `Path(__file__).resolve().parent.parent / "outputs"` anchors the output folder to the script instead of to wherever you are standing.

## 11 The report (ASET component 6)

A calculation that only exists in a terminal is not a deliverable. The last block of the job file assembles the document — and the assembling is almost entirely just adding the results you already have, in the order you want them read.

```
job/beam_B1.py
146 # 6. The report
147 # -----
148 report = Report(
149     title=f"Roof beam B1 -- {job.PROJECT.job_name}",
150     signature=job.PROJECT.signature_block(element="Roof beam B1", revision="A"),
151     preamble=(
152         f"Simply supported roof beam, {job.SPAN / m:.1f} m clear span, {DESIGNATION}. "
153         f"Demands enveloped over {len(env.combinations)} ultimate combinations to "
154         f"AS/NZS 1170.0; capacities to AS 3600:2018."
155     ),
156 )
157 report.add_scope(
158     "Flexural and shear strength of B1 at the ultimate limit state only. "
159     "Deflection, crack control, anchorage and the supporting blockwork are "
160     "outside this calculation."
161 )
162 report.add_assumption(
163     f"Permanent action {job.G_AREAL / kPa:.2f} kPa over a {job.TRIB_WIDTH / m:.1f} m "
164     f"tributary width, plus beam self weight at 24 kN/m3. Imposed action "
165     f"{job.Q_AREAL / kPa:.2f} kPa (AS/NZS 1170.1 Table 3.2, non-trafficable roof) "
166     f"with a {job.Q_PLANT / kN:.0f} kN plant load at {job.PLANT_POSITION / m:.1f} m. "
167     f"Ultimate wind uplift {job.W_WU / kN_per_m:.2f} kN/m from "
168     f"V_des = {job.design_wind_speed():.1f} m/s to AS/NZS 1170.2."
169 )
170 report.add(env.to_calc_result())
171 report.add_figure(f"Shear, moment and deflection under the governing combination "
172                  f"({governing_combo.name})", "B1_diagrams.png")
173 report.add_figure("Moment and shear envelopes over all ULS combinations", "B1_envelope.png")
174 report.add(flexure)
175 report.add(shear)
176 report.add_figure("Section as detailed", "B1_section.png")
177 report.add_conclusion(
178     f"B1 as detailed ({DESIGNATION}) is adequate in flexure "
179     f"(utilisation {flexure.utilisation:.2f}) and in shear "
180     f"(utilisation {shear.utilisation:.2f}) at the ultimate limit state."
181 )
```

**Screenshot 11.1** — a title, a signature block generated from the project record, and then `add()` calls in reading order. The results carry their own inputs, clauses, working and checks, so nothing is transcribed into the report.

### The part only you can write

`add_scope`, `add_assumption` and `add_conclusion` are different from `add`: they take prose, and the library cannot generate them. That is the point. A report that reaches an independent reviewer is read by a person, and a person needs to know what was assumed and what the numbers mean. Keeping the narrative in the same ordered list as the calculations is what stops it drifting to the back of the document where nobody reads it.

outputs/B1\_report.html

# Roof beam B1 -- Northbank Depot

Job: 25-0142   Project: Northbank Depot   Element: Roof beam B1   Revision: A

Result: PASS

NOT VERIFIED FOR ISSUE — one or more modules used in this report have not been checked against the printed standard. Development use only.

Simply supported roof beam, 7.2 m clear span, 350 x 650 | C32 | COV 40 | BOT 3-N24 | LIG N12-2L@250. Demands enveloped over 7 ultimate combinations to AS/NZS 1170.0; capacities to AS 3600:2018.

SCOPE

Flexural and shear strength of B1 at the ultimate limit state only. Deflection, crack control, anchorage and the supporting blockwork are outside this calculation.

ASSUMPTION

Permanent action 5.22 kPa over a 3.6 m tributary width, plus beam self weight at 24 kN/m3. Imposed action 0.25 kPa (AS/NZS 1170.1 Table 3.2, non-trafficable roof) with a 40 kN plant load at 3.0 m. Ultimate wind uplift 3.98 kN/m from V\_des = 41.0 m/s to AS/NZS 1170.2.

## 1. Enveloped demands -- B1 (ULS)

INPUTS

| Symbol | Description           | Value | Unit |
|--------|-----------------------|-------|------|
| n      | Combinations analysed | 7     | -    |

BASIS

- First principles — Linear elastic analysis, results enveloped over load combinations
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states
- AS/NZS 1170.0:2002 Cl 4.2.2 — Combinations for ultimate limit states

RESULTS

| Symbol | Description                     | Value | Unit |
|--------|---------------------------------|-------|------|
| M*_sag | Governing sagging moment [ULS2] | 296.1 | kN.m |
| M*_hog | Governing hogging moment [ULS1] | 0     | kN.m |
| V*     | Governing shear [ULS2]          | 144.2 | kN   |
| delta  | Maximum deflection [ULS2]       | 6.263 | mm   |
| R_A    | Maximum reaction [ULS2]         | 144.2 | kN   |

**Screenshot 11.2** — the top of the generated HTML report. Job header, overall verdict, the not-verified banner, your preamble, and your scope and assumption blocks — then the enveloped demands.

outputs/B1\_report.html

Torsion, lateral instability of slender beams and the effects of sustained load are not considered.

WORKING

| Symbol    | Description                      | Value     | Unit |
|-----------|----------------------------------|-----------|------|
| alpha_2   | Stress block intensity           | 0.85      | -    |
| gamma     | Stress block depth ratio         | 0.826     | -    |
| d_n       | Neutral axis depth               | 86.3      | mm   |
| gamma.d_n | Stress block depth               | 71.28     | mm   |
| d         | Effective depth                  | 586       | mm   |
| d_o       | Depth to outermost tensile layer | 586       | mm   |
| k_u       | d_n / d                          | 0.1473    | -    |
| k_uo      | d_n / d_o                        | 0.1473    | -    |
| z         | Lever arm                        | 550.4     | mm   |
| C_c       | Concrete compressive force       | 6.786e+05 | N    |
| T_s       | Tensile force in reinforcement   | 6.786e+05 | N    |
| A_st      | Tensile steel area               | 1357      | mm^2 |
| M_cr      | Cracking moment                  | 91.31     | kN.m |

RESULTS

| Symbol   | Description                         | Value | Unit |
|----------|-------------------------------------|-------|------|
| phi      | Capacity reduction factor (Class N) | 0.85  | -    |
| M_uo     | Nominal moment capacity             | 373.5 | kN.m |
| phi.M_uo | Design moment capacity              | 317.4 | kN.m |

CHECKS

| Criterion                          | Actual |    | Limit | Unit | Utilisation | Status |
|------------------------------------|--------|----|-------|------|-------------|--------|
| Ductility, k_uo                    | 0.1473 | <= | 0.36  | -    | 0.409       | PASS   |
| Minimum strength, M_uo >= 1.2 M_cr | 373.5  | >= | 109.6 | kN.m | 0.293       | PASS   |
| Flexural strength, M* <= phi.M_uo  | 296.1  | <= | 317.4 | kN.m | 0.933       | PASS   |

astruct.design.as3600.flexure v0.1.0 · UNVERIFIED · not checked · run 2026-09-03T18:41:37+00:00

**Screenshot 11.3** — the flexure check as issued: every intermediate quantity from  $\alpha_2$  through  $d_n$ ,  $k_{uo}$  and the lever arm, then the three checks with utilisations. A reviewer can follow this without opening Python.

|                                                                                                                                                                                                               |             |           |            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-----------|------------|
|                                                                                                                                                                                                               |             |           |            |
| Section as detailed                                                                                                                                                                                           |             |           |            |
| Conclusion                                                                                                                                                                                                    |             |           |            |
| <p><b>CONCLUSION</b></p> <p>B1 as detailed (350 x 650   C32   COV 40   BOT 3-N24   LIG N12-2L@250) is adequate in flexure (utilisation 0.93) and in shear (utilisation 0.37) at the ultimate limit state.</p> |             |           |            |
| Signatures                                                                                                                                                                                                    |             |           |            |
|                                                                                                                                                                                                               | Name        | Signature | Date       |
| Designed                                                                                                                                                                                                      | A. Morrison |           | 2026-09-03 |
| Checked                                                                                                                                                                                                       |             |           |            |

**Screenshot 11.4** — the section drawn to scale from the same object that was checked (so it cannot disagree with it), your conclusion, and a signature block with an empty Checked row. The document knows it has not been checked.

```
python beam_B1.py — last lines
```

```
REPORT  all checks pass: True    issuable: False  
        figures and report written to outputs/
```

**Screenshot 11.5** — and the two flags that matter. `passed: True` means every check passed. `issuable: False` means at least one module in the document is `UNVERIFIED`, and the report says so on its own face.

### PASSED AND ISSUABLE ARE DIFFERENT QUESTIONS

**passed** asks: did the beam satisfy the criteria? **issuable** asks: has the code that evaluated those criteria been checked against the printed Standard? A calculation can pass and still not be issuable, which is exactly the state this library is in today — and the report is built so it cannot pretend otherwise.

## Getting a PDF out

The HTML renderer writes a self-contained page; print it to PDF from any browser, or from the command line with headless Chromium:

```
bash
```

```
$ chrome --headless --print-to-pdf=B1_report.pdf outputs/B1_report.html
```

**Screenshot 11.6** — the same command that produced the document you are reading. The Markdown renderer is there for the other common route: paste into a Word template, or check the `.md` into git and diff calculations between revisions.

## 12 What to do next

### The habits worth carrying over, in order of value

- 1 **One job folder, two files, run from inside it.** Copy the folder for the next job rather than starting blank.
- 2 **Job data first, completely, before any member file.** Loads that live in one place cannot disagree with themselves.
- 3 **Never type a load factor.** If you find yourself writing `1.2 *` anything, the project record is missing information it should have.
- 4 **The design decision is one string.** The same string as the drawing.
- 5 **Read the envelope notes on any module the first time you use it.** They tell you what it does not do.
- 6 **Write the scope, assumptions and conclusion yourself.** The library can generate everything except the engineering.

### The obvious extensions to this job

| Next                                                          | Call                                                                                                                                      |
|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Serviceability — the check this tutorial deliberately skipped | <code>PROJECT.load_combinations(uls=False)</code> , then <code>as3600.effective_stiffness</code> and <code>as3600.check_deflection</code> |
| Crack control and detailing                                   | <code>as3600.check_crack_control</code> , <code>as3600.check_detailing</code> , <code>as3600.check_bar_fit</code>                         |
| Design B2 and B3 as well                                      | A second script importing the same <code>job_data</code> , or a CSV member schedule through <code>design_documentation</code>             |
| Let the search pick the section                               | <code>as3600.minimum_cost_section</code>                                                                                                  |
| Continuous beams                                              | <code>austruct.analysis</code> pattern loading and redistribute — and remember to invert the section for hogging                          |

### Three questions Ferster asks at the end of the ASET paper

They are the right ones to sit with once the mechanics stop being the hard part:

- Which of the six components, if you built it properly, would change your week the most? (For most engineers it is component 2 — project data — not the capacity equations everyone starts with.)
- What data would you have to feed it, and where does that data come from today?
- What is the smallest version of it that would already be useful on the job you are on right now?

#### AND THE STANDING CAVEAT

Nothing in this library has been verified against the printed Standards. Every report it produces says so on its face, and this tutorial does not change that. Treat every number in this document as an illustration of a workflow, not as a design.

---

Tutorial 01 · `tutorials/01_simply_supported_beam/` · Built from the committed job files and captured terminal output by `build_tutorial.py`. Regenerate with `python build_tutorial.py --pdf`.